LeakNix - An Automated Memory Leak Detection and Fixing Tool for Web Applications

Software Project Lab 3 Technical Report

Submitted by
Kamruzzaman Asif
BSSE-1217
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Zerina Begum
Professor
Institute of Information Technology
University of Dhaka

Submission Date: 1 December 2024



Letter of Transmittal

1 December, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **LeakNix** - An Automated Memory Leak Detection and Fixing Tool for Web Applications.

Sir,
I am submitting the technical report for our Software Project Lab 3, "**LeakNix -** An Automated Memory Leak Detection and Fixing Tool for Web Applications."

This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Kamruzzaman Asif
BSSE-1217
Institute of Information Technology
University of Dhaka

1.12.2024
—----------
Supervisor's Signature

Abstract

Most modern web applications leverage frameworks like React, Vue.js, and Angular to accelerate the development of dynamic and engaging user interfaces. These technologies come with a price of unmanaged memory consumption issues, particularly for SPAs. Web memory leaks refer to events in which an application holds onto unnecessary memory without releasing it, thus resulting in excessive use over time.

LeakNix automatically detects and fixes memory leaks in SPAs. Since the detection of memory leaks is complex, LeakNix automatically performs the work of simulating user interactions with a given website and its memory profile in runtime. The different kinds of leaks detected are unreleased objects, leaking event listeners, persistent DOM nodes, and collections, which can be Arrays, Maps, or Sets. Moreover, LeakNix provides automated code fixes by analyzing source code to identify leak patterns and then suggesting patches for those leaks.

Table of Contents

1. Introduction....4
1.1 Motivation....4
1.2 Scope....5
1.3 Project Timeline....5
1.4 Purpose of Document....6
2. Project Description....6
2.1. Quality Function Deployment....6
2.1.1. Normal Requirements....6
2.1.2. Expected Requirements....7
2.1.3. Exciting Requirements....7
2.2 Usage Scenario....7
3. Scenario Based Modeling....8
3.1 Use Case Diagram....9
3.1.1 Level 0....9
3.1.2 Level 1....9
3.1.2 Level 1.1....10
4. Data Based Modeling....14
5. Class Based Modeling....14
5.1 Analysis Classes....14
5.2 Class cards....14
5.3. Class Responsibility and Collaboration Diagram....19
6. Architectural Design....20
6.1 Architectural Context Diagram....20
6.2 Archetypes....21
6.3 Top Level Components....22
6.4 Client-Server Architecture Overview....22
6.5 Mapping Requirements for Software Architecture....23
8. Deployment Diagram....24
9. Preliminary Test Plan....25
9.1 High-Level Description of Testing Goals....25
9.2 Test Cases....25
10. Conclusion....29
11. References....30

1. Introduction

Modern web development heavily depends on frameworks like React, Vue.js, and Angular, which simplify the creation of dynamic and interactive user experiences. However, these frameworks often introduce challenges in managing memory consumption, particularly in Single Page Applications (SPAs). Unlike traditional web applications, SPAs maintain a single runtime, enabling seamless user interactions without full-page reloads. While this improves performance and responsiveness, it also increases the risk of memory leaks—unreleased memory that accumulates over time due to improper resource management.

LeakNix is an automated solution designed to address this critical issue. LeakNix identifies and resolves memory leaks that compromise application performance. Through user interaction simulation and in-depth memory analysis, LeakNix provides detailed reports on detected leaks and also offers code fixes to eliminate them. By doing so, it ensures the stability and efficiency of SPAs, empowering developers to build high-quality web applications in today's demanding development environment.

1.1 Motivation

Up until 2010, most websites were built as Multiple Page Applications (MPAs), requiring full-page reloads for each user request, which slowed performance. The introduction of Single Page Applications (SPAs) improved responsiveness by updating only parts of the page. However, SPAs introduced challenges with memory management. Unlike MPAs, SPAs can remain active for extended periods, during which user interactions and background processes, such as navigation between views or updates to the interface, continuously run. These repeated operations, if not properly managed, can accumulate unnecessary objects that evade the browser garbage collector and thus result in memory leaks, performance degradation, crashes, or even system failures.

```
15  export default class SidePane extends React.Component<SidePaneProps, Side
16      private div = React.createRef<HTMLDivElement>();
17
18      constructor(props: SidePaneProps) {
19          super(props);
20          this.state = {
21              currentPane: this.props.plugins[0],
22          };
23
24          window.addEventListener('hashchange', this.updateStateFromHash);
25      }
26
```

```
23
24          window.addEventListener('hashchange', this.updateStateFromHash);
25      }
26
27      componentDidMount() {
28          this.updateStateFromHash();
29      }
30
31  +   componentWillUnmount() {
32  +       window.removeEventListener('hashchange', this.updateStateFromHash);
33  +   }
```

Figure 1: Memory leak in RoosterJS [1] and its corresponding patch

Several studies have revealed the extent of web memory leaks. For instance, Vilk and Berger reported that over 99% of Google Chrome crashes on low-end Android phones are due to memory problems [2]. They identified more than 50 memory leaks in popular applications, including JavaScript frameworks and Google apps. Another study found public-facing SPAs leaking up to 186 MB per user interaction [3]. The code shown above, taken from Microsoft's roosterjs library, illustrates an example of such a memory leak. Identifying and fixing these leaks is often a time-consuming and tedious task, requiring significant expertise and manual effort, highlighting the need for an automated solution to address memory issues efficiently.

1.2 Scope

The scope of **LeakNix** is given below:

- LeakNix will be an automated memory leak detection and fixing tool.
- It will be a web application.
- It will detect memory leaks in Single Page Applications (SPAs):
    - **Automatically:** Requires minimal manual intervention.
    - **Comprehensively:** Simulates real user interactions to identify various types of leaks, including unreleased objects, event listeners, collections, and persistent DOM nodes.
- LeakNix will provide:
    - **Detailed Reporting:** Identifies the nature, size, and origin of memory leaks.
    - **Automated Code Fixing:** Suggests patches to address detected memory leaks based on the provided source code.

It will enhance application performance, enabling developers to maintain stable and efficient SPAs with ease.

1.3 Project Timeline

The estimated project timeline for **LeakNix** is shown below:

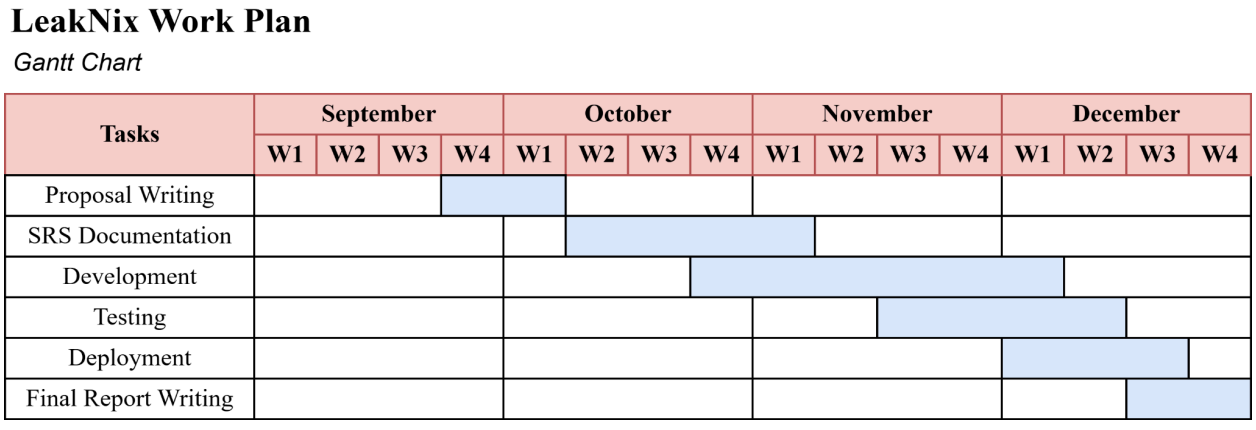


Figure 2: Project Timeline

1.4 Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

The following outlines the journey towards defining the Quality Function Deployment (QFD) for LeakNix. The process began with a thorough examination of relevant academic literature, particularly the works by Vilk and Berger [2] and Shahoor et al. [3]. These provided critical insights into the challenges of detecting and repairing memory leaks in web applications.

This literature review was further enriched by engaging with stakeholders, including developers and engineers from prominent software development firms. These discussions shed light on real-world challenges faced in addressing memory leaks within Single Page Applications (SPAs) and informed the tool's design. Stakeholders highlighted the need for automated solutions that integrate seamlessly into existing workflows while effectively detecting and fixing memory leaks. This collaborative approach ensured that LeakNix's features and functionality were aligned with the practical needs and expectations of its end users.

2.1. Quality Function Deployment

Quality Function Deployment (QFD) is a structured method used to translate customer needs into specific technical requirements for software development. For the LeakNix project, the following requirements are identified:

2.1.1. Normal Requirements

1. **Automated Leak Detection:** LeakNix should automatically detect various types of memory leaks such as unreleased objects, leaking event listeners, persistent DOM nodes, and collections like Arrays, Maps, and Sets.

2. **Browser-Based Simulation:** LeakNix must simulate real user interactions on SPAs and analyze memory profiles in real time to identify persistent memory leaks.
3. **Automated Code Fixing:** The tool should provide basic patches for resolving memory leaks based on the solutions outlined in the referenced research, given the application's source code.

2.1.2. Expected Requirements

1. **Accuracy and Reliability:** LeakNix should demonstrate high accuracy in detecting and fixing memory leaks.
2. **User-Friendly Design:** Configuration, and operation of LeakNix should be intuitive and accessible for developers of varying expertise levels.

2.1.3. Exciting Requirements

1. **Enhanced Reporting:**
    - The tool should generate detailed reports specifying the type, size, and source (optionally) of memory leaks for easy identification by developers.
    - Include heap memory statistics to give developers a clear understanding of memory usage and the impact of detected leaks.
    - Provide a comprehensive report on the fixes applied, detailing the changes made and their expected impact on the application's performance.
2. **Human-in-the-Loop Fixing:** The tool should allow developers to review the generated patches and accept or reject specific fixes before applying them, providing control and flexibility to developers.

2.2 Usage Scenario

"LeakNix: An Automated Memory Leak Detection and Fixing Tool for Web Applications" is a web-based application designed to assist developers in detecting and fixing memory leaks in Single Page Applications (SPAs). It offers automated leak detection, simulation of user interactions, automated code fixing, and detailed reports to ensure efficient memory management. The usage scenario for LeakNix can be outlined as follows:

Access and Setup

Developers can access the LeakNix web application via their browser. They can provide the URL of the SPA for leak detection. Optional settings allow developers to define specific user interaction scenarios for targeted analysis. Additionally, for code remediation and the generation of effective leak patches, developers have the option to upload their source code directly to the

application, streamlining the process of ensuring code quality and enhancement.

Automated Detection of Memory Leaks

LeakNix simulates real user interactions within a browser-based environment to replicate typical application behavior. It analyzes memory usage in real-time, identifying persistent leaks such as unreleased objects, leaking event listeners, DOM nodes, and collections (Arrays, Maps, Sets).

Detailed Reports with Heap Memory Statistics

After the analysis, LeakNix generates a detailed report that includes:

- Types and sizes of detected memory leaks.
- Sources of the leaks (optionally, for easier identification).
- Heap memory statistics, providing an overview of memory consumption patterns and potential areas of optimization.

Automated Fix Generation and Human-in-the-Loop Approach

For detected leaks, LeakNix generates basic patches based on the latest research. Developers can review these patches within the application and decide whether to accept or reject specific changes. Accepted patches are applied to the provided source code, ensuring transparency and developer control.

Iterative Usage for Continuous Monitoring

Developers can use LeakNix periodically to analyze their applications during different stages of development. This iterative approach helps maintain optimal performance and prevents recurring memory issues as the application evolves.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.1.1 Level 0

Name: LeakNix
Primary Actors: Developer
Secondary Actors: Puppeteer Driver

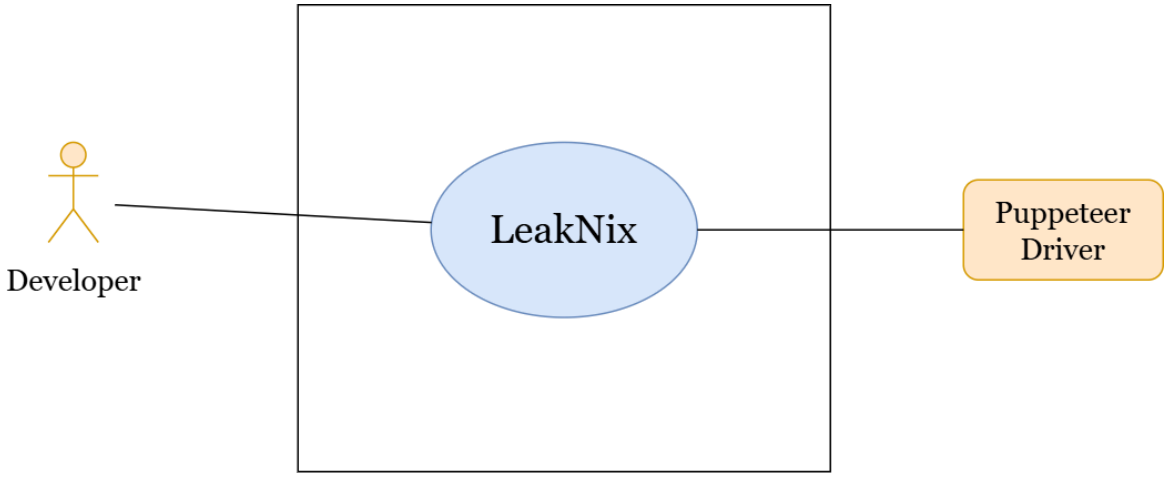


Figure 3: Use Case Level 0

Description of Level 0: The diagram shown above represents the whole LeakNix tool.

3.1.2 Level 1

Name: LeakNix Modules
Primary Actors: Developer
Secondary Actors: Puppeteer Driver

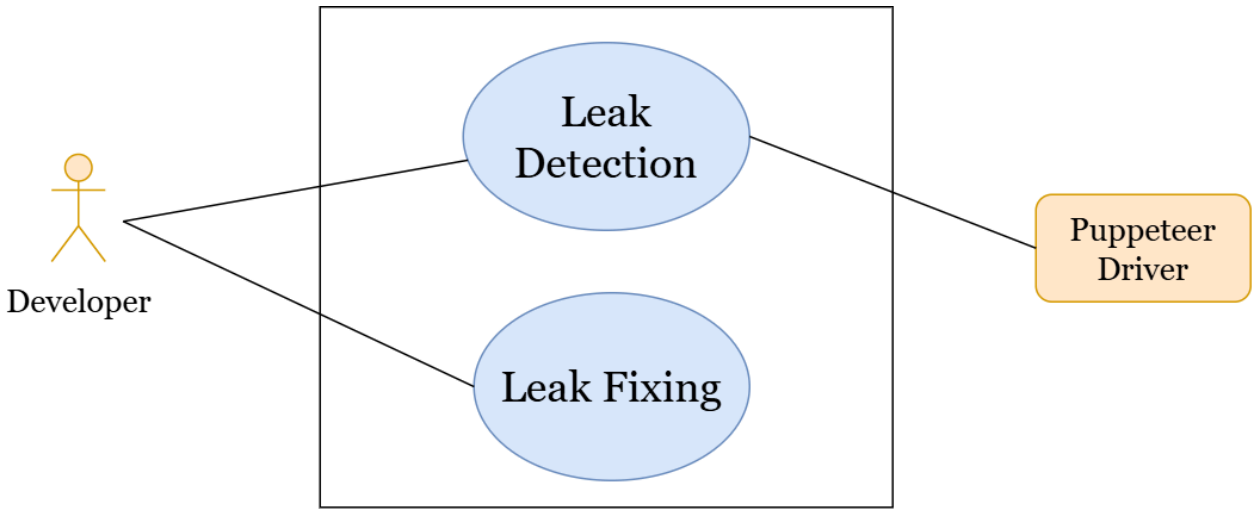


Figure 4: Use Case Level 1

Description of Level 1: The diagram shows core modules of LeakNix which can be elaborated as follows:

1. **Leak Detection:** Detects memory leaks by simulating user interactions, profiling memory usage, analyzing heap snapshots, and generating detection reports.
2. **Leak Fixing:** Resolves detected leaks through source code analysis, automated patch generation, and developer-reviewed fixes, with detailed fix reports.

3.1.2 Level 1.1

Name: LeakNix Modules Elaborated
Primary Actors: Developer
Secondary Actors: Puppeteer Driver

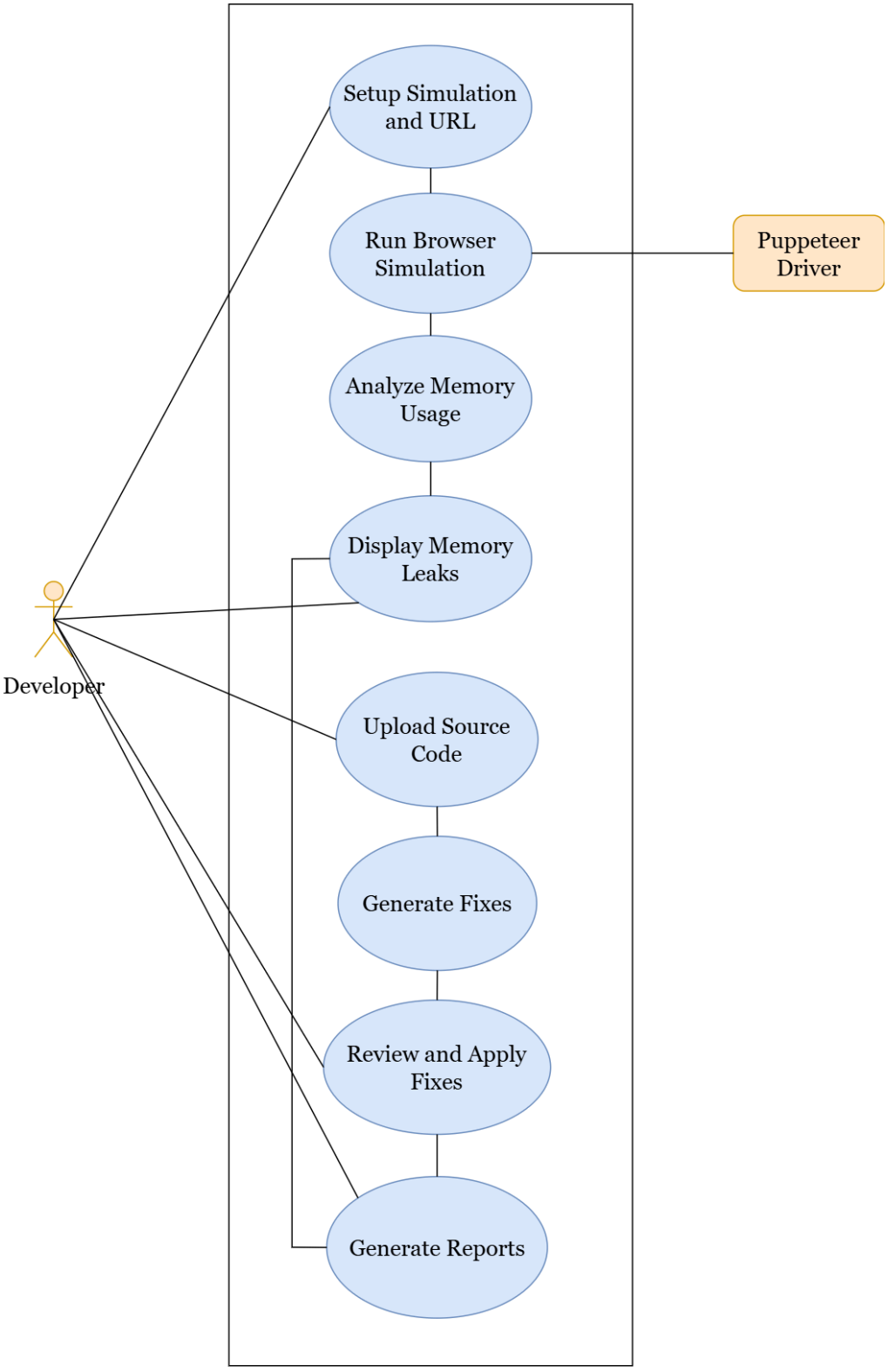


Figure 5: Use Case Level 1.1

Description of Level 1: The diagram shows modules of LeakNix which can be elaborated as follows:

3. **Setup Simulation and URL:** Users start by providing the web application's URL to be analyzed. Additionally, they can upload custom user interaction scenarios, such as specific navigation flows or UI actions, that the system will simulate during memory leak detection.
4. **Run Browser Simulation:** LeakNix launches the web application in a headless browser environment using **Puppeteer** and the **Chrome DevTools Protocol**. The system simulates user interactions, such as button clicks, form submissions, and page navigations, to mimic real usage patterns. During the simulation, it records memory usage across repeated interactions to uncover problematic patterns that may signal leaks.
5. **Analyze Memory Usage:** Once the simulation is complete, LeakNix processes the recorded heap memory profiles. Using advanced analysis, it identifies persistent objects that should have been garbage collected but remain in memory. Specific issues like leaking event listeners, undeleted DOM nodes, and unoptimized collections (e.g., Arrays, Maps, Sets) are flagged for further review.
6. **Display Memory Leaks:** The detected memory leaks are displayed in a clear, interactive dashboard. For each issue, LeakNix highlights the affected memory objects, the potential source of the leak, and its impact on memory consumption. Developers can explore detailed insights, including the specific scenarios that triggered the leaks, enabling them to trace the root cause efficiently.
7. **Upload Source Code:** Before generating fixes, users upload the source code of the web application. LeakNix uses this code to identify patterns contributing to the detected leaks and prepares patches tailored to the application's logic.
8. **Generate Fixes:** LeakNix analyzes the uploaded source code to propose automated patches addressing the detected memory leaks. These patches are based on common solutions outlined in research (e.g., releasing event listeners, clearing timers, and releasing subscriptions). The system prepares these fixes while preserving code readability and adhering to best practices.
9. **Review and Apply Fixes:** Developers can review the generated fixes in a user-friendly interface. For each patch, the tool provides an explanation, such as the root cause of the issue and how the patch resolves it. The developer can accept or reject individual fixes, giving them control over the changes. Once approved, the fixes are applied to the source code automatically, ensuring seamless integration with the project.
10. **Generate Reports:** LeakNix generates comprehensive reports at two key stages:
    - **Detection Report**: Details the detected memory leaks, including their type, size, and impact on performance. It may include heap memory statistics and snapshots to provide a clear understanding of how the leaks affect memory usage during user interactions.

- **Fix Report**: Summarizes the applied fixes, detailing the patches implemented, and the specific leaks addressed. This report helps developers evaluate the impact of the applied fixes and track the resolution of issues over time.

4. Data Based Modeling

No kind of database or data storage system is used for this tool. So this section does not contain any relevant analysis.

5. Class Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

1. MemoryAnalyzer
2. ScenarioBuilder
3. Test
4. SimulationEngine
5. MemoryProfiler
6. DetectionReportGenerator
7. CodeProcessor
8. ASTParser
9. PatchGenerator
10. FixManager
11. FixReportGenerator

5.2 Class cards

MemoryAnalyzer	
Attributes	**Methods**
-heapSnapshots -detectedLeaks -analysisResults	+analyzeHeapSnapshots(heapSnapshots) -findCollectionLeaks() -findEventListenerLeaks() -findObjectLeaks() -findDOMNodeLeaks()

	+generateAnalysisResults(detectedLeaks)
Responsibilities	**Collaborators**
• Central class responsible for analyzing heap snapshots. • Detects four types of memory leaks: collections, event listeners, unreleased objects, and persistent DOM nodes. • Generates results for leak detection.	SimulationEngine, DetectionReportGenerator

ScenarioBuilder	
Attributes	**Methods**
-url -customScenario	+createTests() -addCustomScenario(customScenario)
Responsibilities	**Collaborators**
• Creates test scenarios based on user-provided URLs or custom inputs.	Test

Test	
Attributes	**Methods**
-testScenarios -validatedTestScenarios	+validateTests(testScenarios) +getValidatedTests()
Responsibilities	**Collaborators**
• Validates the test scenarios created by ScenarioBuilder. • Prepares valid tests for the SimulationEngine.	ScenarioBuilder, SimulationEngine

SimulationEngine	
Attributes	**Methods**

-tests -heapSnapshots -memoryProfilerInstance	+runSimulation(tests) -initBrowser() -setupEnvironment() -closeBrowser() +getHeapSnapshots()
Responsibilities	**Collaborators**
● Runs the test scenarios. ● Captures heap snapshots through MemoryProfiler during test execution. ● Provides the captured heap snapshots to MemoryAnalyzer	Test, MemoryProfiler, MemoryAnalyzer

MemoryProfiler	
Attributes	**Methods**
-capturedSnapshots	+captureHeapSnapshot() -clearSnapshots() +getSnapshots()
Responsibilities	**Collaborators**
● Captures memory heap snapshots during test execution.	SimulationEngine

DetectionReportGenerator	
Attributes	**Methods**
-leakResults -reportData	+generateReport(leakResults) -formatDetectionResults()
Responsibilities	**Collaborators**
● Generates detailed reports based on results from MemoryAnalyzer. ● Formats and organizes the detected leaks into developer-friendly reports.	MemoryAnalyzer

CodeProcessor	
Attributes	**Methods**
-sourceCode	+validateCode() +getSourceCode()
Responsibilities	**Collaborators**
● Accepts the application's source code from the user. ● Validates the source code for issues that can prevent patching. ● Provides validated source code to ASTPasrser and FixManager.	ASTParser, FixManager

ASTParser	
Attributes	**Methods**
-abstractSyntaxTree -parsingResults	+generateAST(sourceCode) +extractStructuralInfo()
Responsibilities	**Collaborators**
● Converts source code into an Abstract Syntax Tree (AST). ● Extracts the structural and logical information needed for fixes.	CodeProcessor, PatchGenerator

PatchGenerator	
Attributes	**Methods**
-patches -detectedIssues	+generateFixes(AST) -findLeakingPatterns() -applyFixPatterns() +mapIssuesToFixes(detectedIssues)
Responsibilities	**Collaborators**
● Uses AST to generate fixes.	ASTParser, FixManager

• Find leaking patterns and apply relevant fixes. • Maps issues to fixes.	

FixManager	
Attributes	**Methods**
-patchCollection -appliedFixes	+reviewFixes() +applyFixes(acceptedFixes)
Responsibilities	**Collaborators**
• Initiates issues and fixes for review. • Applies fixes after developer acceptance (Human-in-the-loop).	PatchGenerator, FixReportGenerator

FixReportGenerator	
Attributes	**Methods**
-fixResults -reportData	+generateFixReport(fixResults)
Responsibilities	**Collaborators**
• Generates a detailed report of applied fixes.	FixManager

5.3. Class Responsibility and Collaboration Diagram

The CRC diagram for LeakNix is given below -

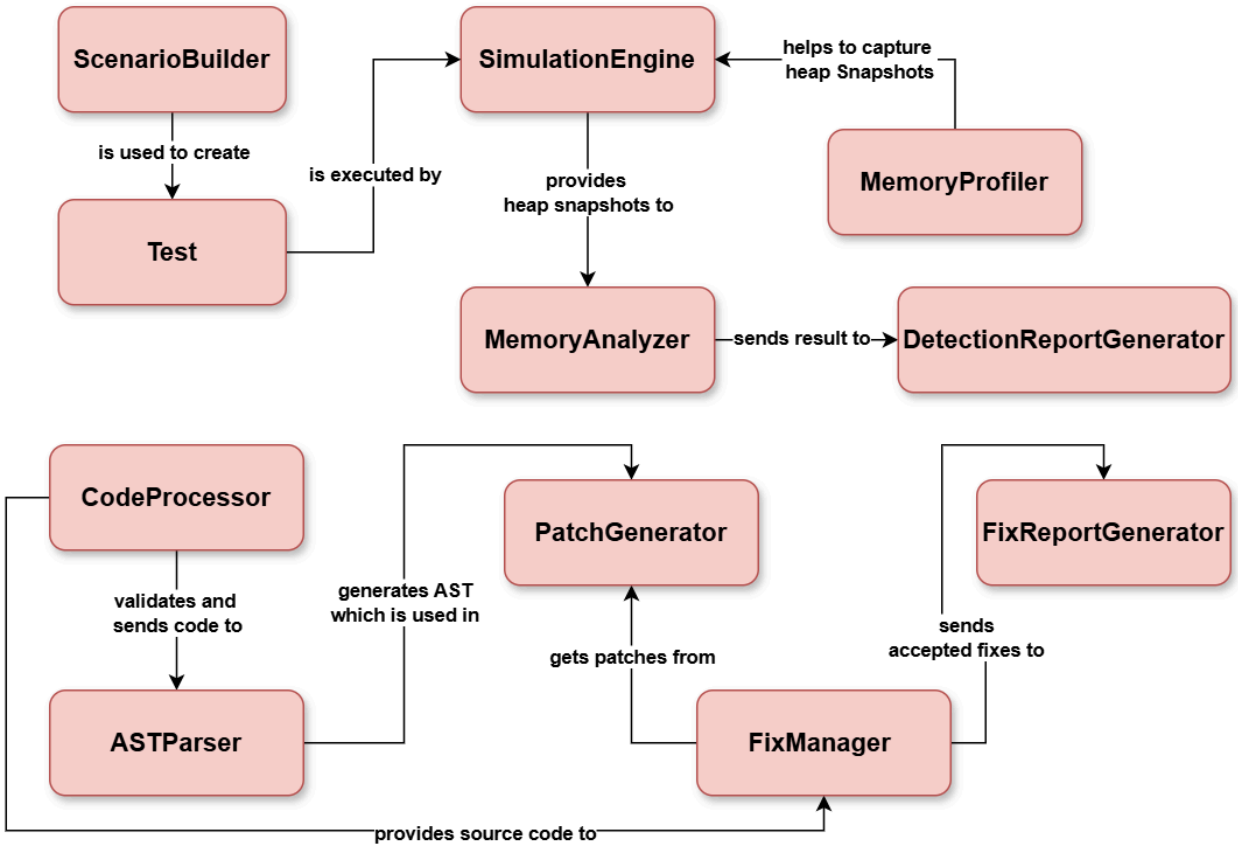


Figure 6: CRC Diagram of LeakNix

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

At the architectural design level, a software architect uses an architectural context diagram (ACD) to model the manner in which software interacts with entities external to its boundaries. Systems that interoperate with the target system are represented as -

- **Superordinate systems**— Those systems that use the target system as part of some higher-level processing scheme.
- **Subordinate systems**— Those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.
- **Peer-level systems**— those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system.
- **Actors**— entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing.

Each of these external entities communicates with the target system through an interface.

Representation of LeakPair in Architectural Context Diagram:

- Superordinate systems - Our system does not have any superordinate system.
- Subordinate systems - Our system uses the Puppeteer Driver Library to simulate user interactions.
- Peer-level systems - Our system does not have any peer-level system.
- Actors - Developers produce and consume information.

The architectural context diagram for the tool is given below-

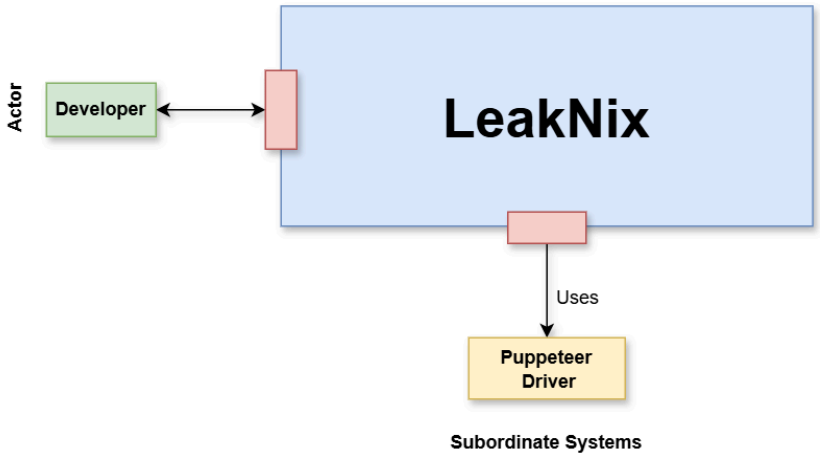


Figure 7: Architectural Context Diagram

6.2 Archetypes

We have derived the archetype by examining the analysis classes. We have found the following archetype:

Client-Server Architecture Pattern

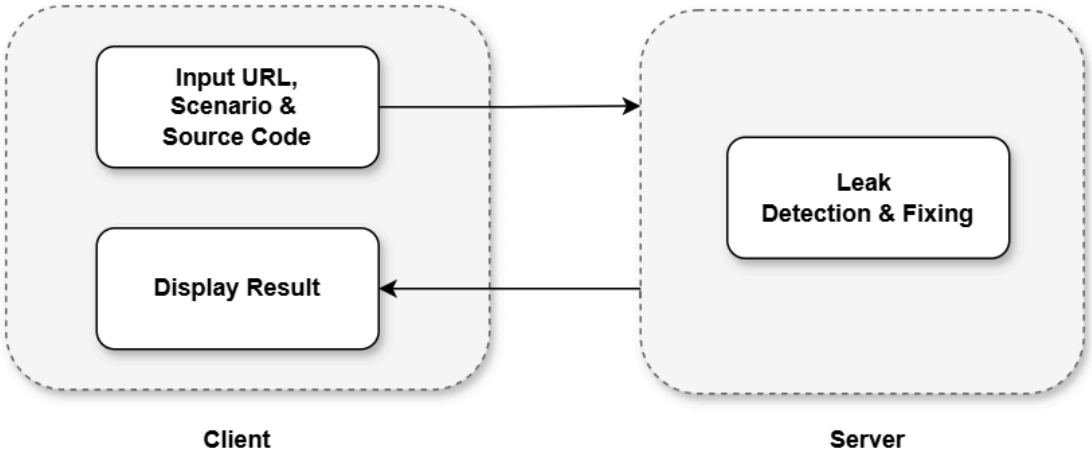


Figure 8: Archetypes

The Client-Server Architecture Pattern divides tasks between clients (requesters) and servers (providers). Clients send requests, and servers process them. This model is common in web applications and email services.

6.3 Top Level Components

The overall architectural structure with top-level components is illustrated below-

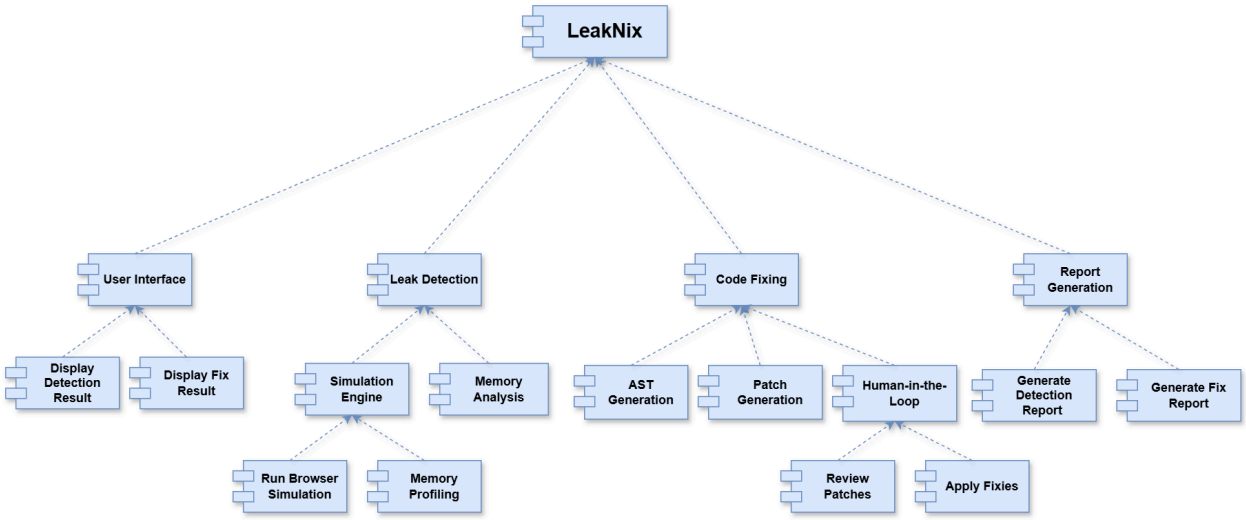


Figure 9:Top Level Components

6.4 Client-Server Architecture Overview

This section provides an overview of the Client-Server Architecture, highlighting its key components and interactions.

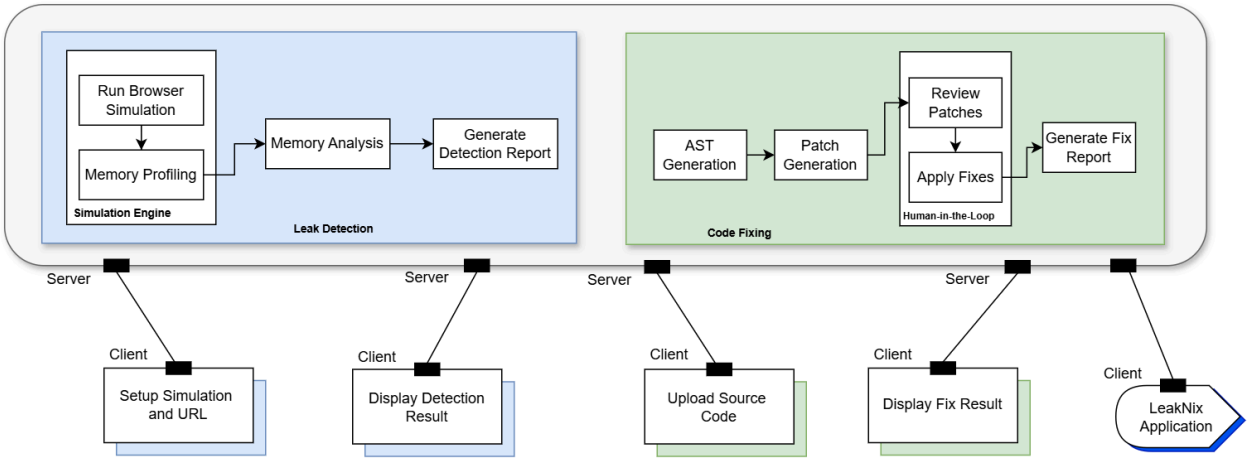


Figure 10: Client Server Architecture Overview

The system utilizes a client-server architecture:

Detection Phase

- The user triggers this event by setting up the simulation and inputting the URL.
- The server simulates user interactions and profiles memory usage.
- Memory analysis identifies potential leaks and generates a detection report.

Fixing Phase

- The user triggers this event by uploading the Source code.
- Source code is parsed into an AST.
- Patches are generated based on detected leaks.
- Human experts review and refine patches.
- Approved patches are applied to the source code.

6.5 Mapping Requirements for Software Architecture

The mapping of the derived components with the requirements is outlined below. The table demonstrates how each top-level component of the system contributes to fulfilling specific requirements:

Requirements	Components
Automated Leak Detection	Run Browser Simulation, Memory Profiling, Memory Analysis
Browser-Based Simulation	Run Browser Simulation, Simulation Engine
Automated Code Fixing	AST Generation, Patch Generation, Apply Fixes
Human-in-the-Loop Fixing	Review Patches, Display Fix Result, Apply Fixes
Enhanced Reporting	Generate Detection Report, Generate Fix Report

8. Deployment Diagram

A UML deployment diagram focuses on the structure of the software system and is helpful in showing the physical distribution of a software system among hardware platforms and execution environments. Deployment diagrams are typically used to visualize the physical hardware and software of a system. Using it you can understand how the system will be physically deployed on the hardware. Deployment diagrams help model the hardware topology of a system compared to other UML diagram types which mostly outline the logical components of a system.

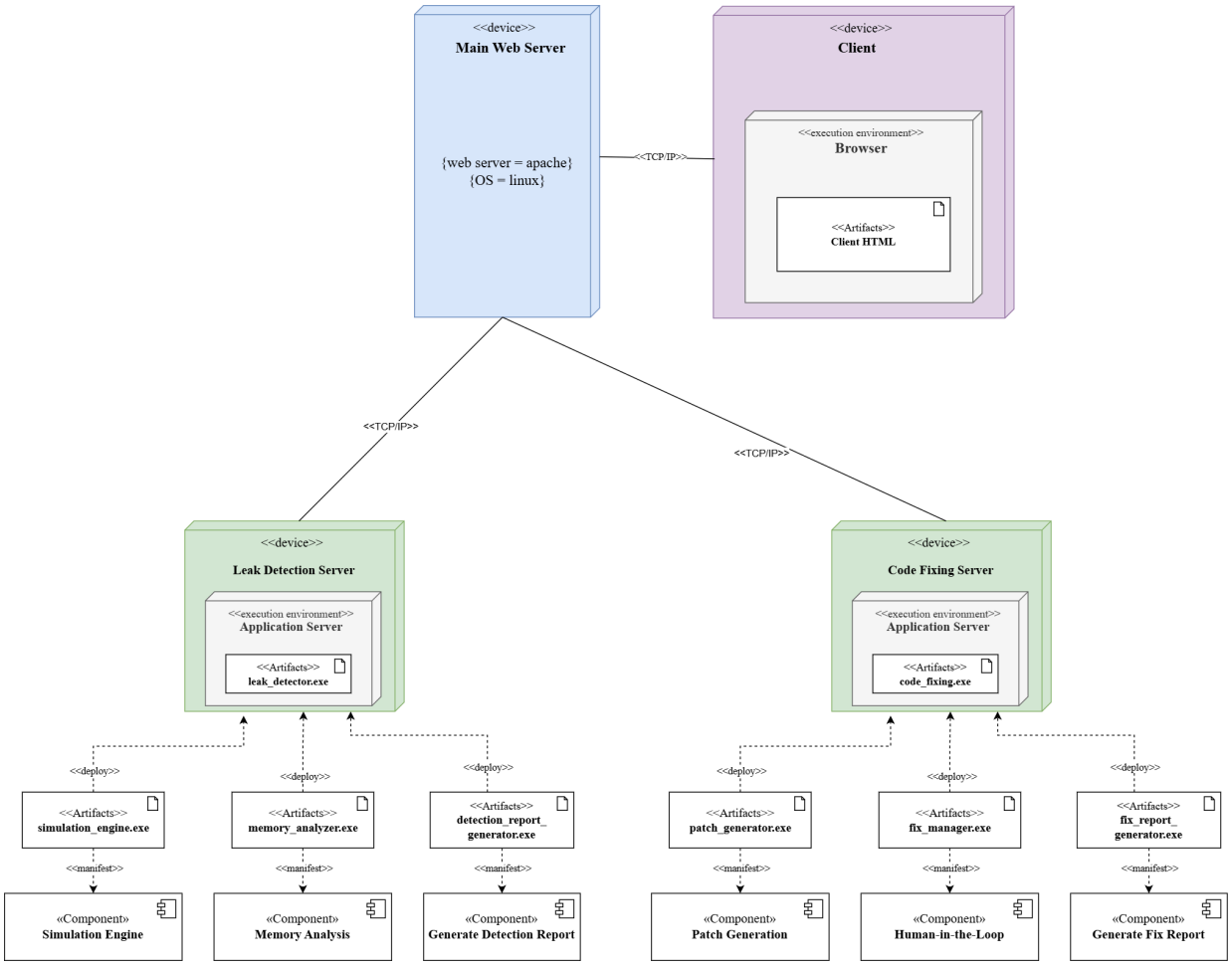


Figure 11: Deployment Diagram

9. Preliminary Test Plan

This chapter provides a high-level overview of the testing goals and a summary of the features to be tested for the LeakNix tool.

9.1 High-Level Description of Testing Goals

The primary goals of testing LeakNix are as follows:

- To demonstrate that the tool meets its specified requirements.
- To ensure that memory leaks are accurately detected in single-page applications (SPAs).
- To verify that browser simulations correctly execute user interactions and generate memory profiles.
- To validate the accuracy of heap analysis in identifying persistent memory issues.
- To confirm that the source code analysis generates correct and appropriate patches for fixing memory leaks.
- To ensure that the tool provides detailed and comprehensible reports for both detection and fixing stages.
- To test the user interface for seamless navigation and interaction across all modules.
- To ensure the tool operates smoothly without errors or crashes during various workflows.

9.2 Test Cases

Test cases for the testing of this tool are documented below:

Test Case ID	T1
Title	Setup Simulation and URL
Test Steps	1. Input the web application URL. 2. Upload a user-defined interaction scenario.
Expected Outcome	● The URL is accepted without errors. ● Interaction scenarios are uploaded successfully and ready for simulation.

Test Case ID	T2
Title	Run Browser Simulation
Test Steps	1. Run the browser simulation module. 2. Check the memory profiles generated during the

	simulation.
Expected Outcome	● The browser launches and interacts with the web application as specified. ● Accurate memory profiles are captured without omissions or errors.

Test Case ID	T3
Title	Analyze Memory Usage
Test Steps	1. Run the analysis module on the collected memory profiles. 2. Inspect the identified memory leaks.
Expected Outcome	● The tool identifies all persistent memory leaks. ● Each leak is categorized correctly (e.g., unreleased objects, event listeners).

Test Case ID	T4
Title	Display Memory Leaks
Test Steps	1. Navigate to the memory leak results page. 2. Review the visualized results and details.
Expected Outcome	● Results are displayed clearly and are user-friendly with relevant details like type, size, and source.

Test Case ID	T5
Title	Upload Source Code
Test Steps	1. Upload the source code of the application. 2. Proceed to the fix generation step.
Expected Outcome	● The source code is uploaded without errors. ● The tool confirms readiness for fix generation.

Test Case ID	T6
Title	Generate Fixes
Test Steps	1. Run the fix generation module. 2. Inspect the patches generated.

Expected Outcome	● The tool generates accurate and applicable patches for identified leaks.

Test Case ID	T7
Title	Review and Apply Fixes
Test Steps	1. Navigate to the fix review page. 2. Accept or reject individual fixes. 3. Apply accepted fixes.
Expected Outcome	● The interface displays fixes with clear explanations. ● Accepted fixes are applied successfully, and the code updates correctly.

Test Case ID	T8
Title	Generate Reports
Test Steps	1. Run the tool for detection 2. Run the tool for fixing
Expected Outcome	● The detection report summarizes memory leaks by type, size, and source. ● The fixing report details applied patches and the expected impact on memory usage.

Test Case ID	T9
Title	Handle Invalid URL
Test Steps	1. Input an invalid or malformed URL. 2. Attempt to proceed with simulation setup.
Expected Outcome	● The tool rejects invalid URLs with a clear error message.

Test Case ID	T10
Title	No Interaction Scenario Provided
Test Steps	1. Input a web application URL without uploading a custom scenario. 2. Run the simulation with default settings.

Expected Outcome	● The tool runs the simulation using predefined interactions and captures memory profiles successfully.

Test Case ID	T11
Title	Handle Unsupported Source Code
Test Steps	1. Upload source code in an unsupported language or structure. 2. Proceed to fix generation.
Expected Outcome	● The tool rejects unsupported source code with a detailed error message.

This test plan ensures that LeakNix's functionality is thoroughly validated across all modules, covering both expected behaviors and edge cases.

10. Conclusion

In the present report, I have outlined the most critical technical specifications that will be necessary for developing LeakNix: Automated Memory Leak Detector and Fixer for Single Page Applications (SPAs). The scope, assumptions, Quality Function Deployment (QFD), and usage scenarios have been studied in order to outline the functional characteristics of the tool. To encompass the system's workflow, modeling techniques such as scenario-based modeling and data portraying were performed owing to its complexity.

The architectural design was supported by its core components as well as the interactions of the components. Also included, is a description outlining the aims of the tests and a summary of the tests performed, to ensure the effectiveness and functionality of the tool. For better clarity and understanding of the technical issues involved, figures and diagrams have been included in the report.

LeakNix is an innovative step aimed at addressing the long-standing challenge of memory leaks, particularly in SPAs with the capability to automatically detect and resolve pending issues. Probably in a growing number of future developments, the primary direction of such evolutionary extension could be: consistent support of a wider variety of application frameworks, improvement of existing algorithms for fixing problems, and enhancement of the level of end-user reporting within the application.

In this document, I will try to present the technical tools of LeakNix and the processes that occur during this particular system. I am confident that this report will give the readers a concise and clear understanding of the matter at hand.

11. References

1. "roosterjs." [Online]. Available: https://github.com/microsoft/roosterjs/commit/c3f2f0c4d229502c634e6c99b604df3e5f47b9b6. Accessed: October. 5, 2024.
2. J. Vilk and E. D. Berger, “Bleak: Automatically debugging memory leaks in web applications,” Commun. ACM, vol. 63, no. 11, p. 146–153, Oct 2020. [Online]. Available: https://doi.org/10.1145/3422598
3. Shahoor, A., Khamit, A. Y., Yi, J., & Kim, D. (2023). LeakPair: Proactive Repairing of Memory Leaks in Single Page Web Applications. arXiv. https://arxiv.org/abs/2308.08144